Tutorial No : 9

Tutorial Title: Represent graphs and generate minimum spanning trees using Prim's algorithm.
Name of Student: Shweta Lohakare
Class: SY **Div:** 2 **PRN No:** B25ET1099 **Batch:** B2

Program:

```
#include <stdio.h>
#define INF 9999
int main()
{   int n, cost[10][10], visited[10] = {0};
    int edges = 0, minCost = 0;
    int i, j, min, a, b;
   printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (0 if no edge):\n");
   for (i = 0; i < n; i++)
 {    for (j = 0; j < n; j++)
     {     scanf("%d", &cost[i][j]);
    if (i != j && cost[i][j] == 0)
              cost[i][j] = INF;     }    }
   visited[0] = 1;
    printf("\nEdges in Minimum Spanning Tree:\n");
   while (edges < n - 1)
 {   min = INF;
       a = -1;
       b = -1;
     for (i = 0; i < n; i++)
 {    if (visited[i])
 {    for (j = 0; j < n; j++)
 {     if (!visited[j] && cost[i][j] < min)
 {    min = cost[i][j];
       a = i;
      b = j;  }
           }
         }
       }
 if (a == -1)
```

```
{      printf("MST is not possible. Graph is disconnected.\n");
          return 0;     }
 printf("%d - %d : %d\n", a, b, min);
    minCost = minCost + min;
      visited[b] = 1;
      edges++;    }
printf("\nTotal minimum cost = %d\n", minCost);
return 0; }
```

Output:

main.c

```c
#include <stdio.h>
#define INF 9999
int main()
{   int n, cost[10][10], visited[10] = {0};
    int edges = 0, minCost = 0;
    int i, j, min, a, b;
   printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (0 if no edge):\n");
   for (i = 0; i < n; i++)
{    for (j = 0; j < n; j++)
     {     scanf("%d", &cost[i][j]);
    if (i != j && cost[i][j] == 0)
               cost[i][j] = INF;     }    }
   visited[0] = 1;
    printf("\nEdges in Minimum Spanning Tree:\n");
   while (edges < n - 1)
{   min = INF;
       a = -1;
       b = -1;
      for (i = 0; i < n; i++)
{    if (visited[i])
{    for (j = 0; j < n; j++)
{     if (!visited[j] && cost[i][j] < min)
{    min = cost[i][j];
       a = i;
      b = j;  }
           }
         }
       }
  if (a == -1)
{      printf("MST is not possible. Graph is disconnected.\n");
          return 0;     }
   printf("%d - %d : %d\n", a, b, min);
      minCost = minCost + min;
       visited[b] = 1;
       edges++;    }
  printf("\nTotal minimum cost = %d\n", minCost);
  return 0; }
```

Output

```
Enter number of vertices: 4
Enter adjacency matrix (0 if no edge):
0 2 6 0
2 0 3 8
6 3 0 5
0 8 5 0


Edges in Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 3
2 - 3 : 5

Total minimum cost = 10
```